1. Paralelización de tareas

La ejecución de tareas puede acelerarse aprovechando los procesadores disponibles en el sistema. La ejecución secuencial de las tareas se sustituye por varios procesos (o hilos) que trabajan de zona simultánea. La paralelización es conveniente en tareas de cómputo intensivo que realizadas en un único proceso tendrían tiempos de ejecución muy elevados. Si las tareas necesitan acceder a regiones de memoria compartida, está más indicado crear varios hilos de ejecución. Si están bien diferenciadas y son independientes entre sí es mejor usar procesos, ya que cada proceso dispone de su propio espacio de memoria. En sistemas con varios nodos es más recomendable usar procesos.

En esta sección se discute la paralelización de tareas (en Python y R) usando procesos en distintos sistemas:

- Windows 11 Home
- Ubuntu 20.04.6 LTS (Curie)
- CentOS Linux release 8.3.2011 (Magerit-3)

1.1. Paralelización en R

Se usan los paquetes ***foreach*** y ***doParallel*** en Windows. En Linux se usa el paquete ***Rmpi [2]***.

1.1.1. Paralelización en Windows

Se paralelizan los bucles en los que las iteraciones sean de cómputo intensivo e independientes entre sí, siguiendo los siguientes pasos:

1. Determinar el número de trabajadores paralelos: la función ***detectCores*** devuelve el número de CPUs en el sistema, que podrán aprovecharse para la paralelización. Conviene dejar al menos un núcleo disponible para otras tareas del sistema, lo que evita sobrecargas. Por lo tanto, al resultado devuelto por ***detectCores*** se le suele restar 1.
2. Usar la función ***makeCluster*** para configurar el entorno de trabajo paralelo.
3. Usar la función ***registerDoParallel*** para que todos los bucles que utilicen la instrucción ***foreach*** lo hagan según la configuración establecida anteriormente.
4. Realización paralela de los bucles usando ***foreach*** combinado con ***%dopar%***, según la sintaxis:

   res <- foreach (i = 1:n, .combine=combineFun) %dopar% { #Code }

   donde:

   - ***res*** es el resultado devuelto por el bucle después de aplicar la función de combinación sobre las salidas de las iteraciones.
   - ***%dopar%*** especifica que se va a ejecutar el bucle de forma paralela según la configuración establecida.

- ***combineFun*** es la función de combinación especificada para el argumento ***.combine***. Especifica cómo deben combinarse los resultados de cada iteración. Puede ser una función predefinida o una función definida por el usuario. Por ejemplo, el argumento ***"rbind"*** combina los datos en un *dataframe* por filas, donde cada fila es el *output* de cada iteración.

Es importante destacar que si el bucle utiliza librerías específicas, será necesario volverlas a cargar, aunque hubieran sido cargadas previamente al inicio del *script*. Esto es porque cada proceso tiene su propio entorno de ejecución, distinto del entorno global.

5. Llamar a ***stopCluster*** para que termine los procesos en ejecución y libere los recursos utilizados.

1.1.2. Paralelización en Linux

Los procesos se crean antes de la ejecución del *script*:

- Curie: con el mandato desde línea de comandos:

mpirun -np ntasks Rscript script.R

donde *ntasks* define el número de procesos que ejecutarán *script.R*.

- Magerit-3: con un archivo de *shell* .sh. El número de trabajadores simultáneos se especifica en las directivas ***#SBATCH –nodes=nnodes*** y ***#SBATCH –ntasks=n***, donde *nnodes* es el número de nodos y *n* es el número total de tareas. La directiva ***#SBATCH –tasks-per-node*** puede usarse para indicar el número de tareas que se ejecutan en cada nodo.

  En el .sh deben constar los siguientes mandatos:

  1. ***module load R/#version*** donde la #version es la versión de R a utilizar. El mandato ***module spider R*** devuelve las versiones disponibles de R en el *cluster*.
  2. ***mkdir -p /carpeta/de/archivos/temporales*** para crear una carpeta en la que se guardarán los archivos temporales generados durante la ejecución.
  3. ***srun –ntasks=$SLURM_NTASKS Rscript /ruta/al/script.R*** donde *ntasks* define el número de veces que se lanzará el *script* de R.
  4. ***rm -rf /carpeta/de/archivos/temporales*** para eliminar el contenido de la carpeta de archivos temporales.

  En el código 1 se presenta un ejemplo con el contenido de este archivo.

La paralelización de las tareas se hace usando el estándar MPI. Cuando se ejecuta ***mpirun*** o ***srun***, se crea un grupo de procesos que pueden intercambiar mensajes entre sí por medio de un comunicador. Por defecto, se les asigna el comunicador 0, aunque el paquete ***Rmpi*** permite crear comunicadores nuevos para grupos de procesos. A continuación, se indican algunas funciones que resultan útiles:

- ***mpi.comm.size*** devuelve el número de procesos en un comunicador.
- ***mpi.comm.rank*** devuelve el identificador del proceso dentro de un grupo de procesos.

- ***mpi.send.Robj*** envía un objeto de R (por ejemplo, un *dataframe*) a otro proceso dentro de un comunicador. Utiliza la función ***serialize*** para convertir el objeto a una cadena de caracteres binarios.
- ***mpi.recv.Robj*** recibe un objeto de R desde otro proceso dentro de un comunicador. Usa ***unserialize*** para regenerar el objeto.
- ***mpi.any.source*** especifica que un mensaje puede llegar desde cualquier origen. Es de gran utilidad en el argumento ***source*** de ***mpi.recv.Robj***.

1.2. Paralelización en Python

Se usa el paquete ***concurrent.futures*** en Windows. En Linux se usa el paquete ***mpi4py [1]***.

1.2.1. Paralelización en Windows

Se paralelizan los bucles en los que las iteraciones sean de cómputo intensivo e independientes entre sí, siguiendo los siguientes pasos:

1. Determinar el número de trabajadores paralelos: la función ***os.cpu_count*** devuelve el número de CPUs en el sistema.
2. Crear un manejador de contexto que asegure la gestión segura y eficiente de los procesos. Se usa la sintaxis:

   with ProcessPoolExecutor(max_workers = num_workers - 1) as executor:

   donde ***num_workers - 1*** define el número de procesos.
3. Usar la función ***map*** para aplicar una función a un iterable de manera concurrente. Las iteraciones estarán definidas en una función en vez del cuerpo del bucle. Cada elemento del iterable contiene los argumentos de la función. El resultado de ***map*** es un elemento iterable con los valores devueltos en cada iteración. La sintaxis se puede resumir de la siguiente forma:

   for res in executor.map(iterFun, args):

   donde:

   - ***res*** es el iterable devuelto por ***map***.
   - ***iterFun*** es la función que se ejecuta en cada iteración del bucle.
   - ***args*** es una lista con los argumentos de cada iteración. Cada argumento solo puede contener un único objeto, por lo que es recomendable usar una tupla con todos los argumentos y descomprimirla al inicio de ***iterFun***.

1.2.2. Paralelización en Linux

Los procesos se crean antes de la ejecución del *script*:

- Curie: con el mandato desde línea de comandos:

  mpirun -np ntasks python script.py

- Magerit-3: con un archivo de *shell* .sh.

  En el .sh deben constar los siguientes mandatos:

  1. ***module load Anaconda3/#version***
  2. ***source ~/.bashrc***
  3. ***conda activate #entorno*** donde #entorno es el entorno Conda que se desea cargar.
  4. ***mkdir -p /carpeta/de/archivos/temporales***
  5. ***srun –ntasks=$SLURM_NTASKS python /ruta/al/script.py***
  6. ***rm -rf /carpeta/de/archivos/temporales***

  En el código 2 se presenta un ejemplo con el contenido de este archivo.

La paralelización de las tareas se hace usando el estándar MPI. A continuación, se indican algunas funciones que resultan útiles:

- ***MPI.COMM_WORLD*** obtiene el comunicador predeterminado.
- ***Get_size*** devuelve el número de procesos en un comunicador.
- ***Get_rank*** devuelve el identificador del proceso dentro de un grupo de procesos.
- ***send*** envía un objeto a otro proceso dentro de un comunicador.
- ***recv*** recibe un objeto desde otro proceso dentro de un comunicador.
- ***MPI.ANY_SOURCE*** especifica que un mensaje puede llegar desde cualquier origen.

1.3. Evaluación de la paralelización en Linux

Asegurarse de que la paralelización se haya realizado exitosamente es imprescindible. De lo contrario, el sistema podría estar sobrecargado y tardar más de lo previsto en realizar el trabajo.

La herramienta ***htop*** sirve para el monitoreo del sistema en tiempo real y resulta de gran utilidad. La figura 1 muestra un posible resultado de la ejecución de ***htop***. Se diferencian 2 zonas:

- Zona superior: proporciona una vista general y resumida del estado del sistema:
  - Uso de recursos: muestra unas barras con el uso de cada CPU disponible en el sistema. Asimismo, unas barras adicionales que indican el uso de memoria RAM y espacio de *swap*.

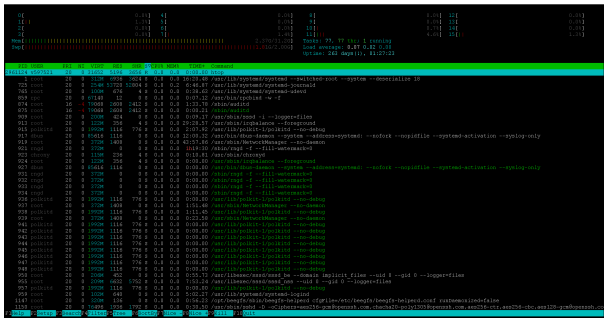


Figura 1: Posible resultado de la ejecución de *htop*.

- Tareas: indica el número de procesos en el sistema, de hilos, y los procesos que están ejecutándose.
- Carga del sistema: indica el promedio de los procesos en ejecución o en espera en los últimos 1, 5 y 15 minutos. Si estos valores son mayores que el número de CPUs, el sistema está sobrecargado (varios procesos compiten por los mismo recursos). Esta situación es indeseable porque ralentiza la ejecución de los trabajos (los cambios de contexto y los accesos a disco si se llena la RAM son algunas de las causas de esta situación).
- Tiempo de actividad: tiempo que ha estado en funcionamiento el sistema desde el último reinicio.

- Zona inferior: presenta información detallada de los procesos del sistema como el porcentaje de CPU utilizada (CPU %), el estado del proceso (S) o el comando que inició el proceso (*Command*). Además, permite realizar operaciones como filtrar, ordenar o mandar señales a los procesos.

En Curie, el monitoreo de recursos utilizando *htop* es directo. Sin embargo, Magerit-3 está formado por varios nodos, por lo que será necesario ejecutar la herramienta en los nodos correspondientes. A continuación, se describe un procedimiento para monitorear recursos en Magerit-3:

1. Ejecutar ***sacct*** y anotar el identificador del trabajo *JobID*.
2. Ejecutar ***scontrol show job #JobID*** para obtener información detallada del trabajo. En el campo *NodeList* aparecen los nodos en los que se está ejecutando el trabajo.
3. Ejecutar ***ssh #Node*** para conectarse a un nodo de los que aparecen en *NodeList*.
4. Ejecutar ***htop*** para ver la utilización de recursos en dicho nodo.

```
#!/bin/bash
##----------------------- Start job description ----------------------
#SBATCH --partition=standard
#SBATCH --job-name=multi_Grace
#SBATCH --nodes=1
#SBATCH --ntasks=40
#SBATCH --mem-per-cpu=4096
#SBATCH --time=50:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=@mail_address
#SBATCH --output=out-%j.log
#SBATCH --error=err-%j.log
#SBATCH --chdir=/home/v597/v597521/multigrace/
##------------------------ End job description ------------------------

#Load R version
module load R/4.2.2-foss-2022b

#Create scratch directory for temporary files
export SCRATCH="/home/v597/v597521/multigrace"
mkdir -p $SCRATCH/$SLURM_JOB_ID

#Run job
srun --ntasks=$SLURM_NTASKS Rscript /home/v597/v597521/multigrace/
   multigrace_par_mpi.R

#Remove files from scratch directory
rm -rf $SCRATCH/$SLURM_JOB_ID
```

Código 1: Código .sh para lanzar un trabajo en R en Magerit-3. Se solicitan un total de 40 tareas distribuidas en 1 único nodo. El tiempo máximo de ejecución del trabajo es de 50 horas.

```
#!/bin/bash
##----------------------- Start job description ----------------------
#SBATCH --partition=standard
#SBATCH --job-name=multi_Leaspy
#SBATCH --nodes=1
#SBATCH --ntasks=40
#SBATCH --mem-per-cpu=4096
#SBATCH --time=50:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=@mail_address
#SBATCH --output=out-%j.log
#SBATCH --error=err-%j.log
#SBATCH --chdir=/home/v597/v597521/multileaspy/
##------------------------ End job description ------------------------

#Load conda environment
module load Anaconda3/2024.02-1
source ~/.bashrc
conda activate leaspy

#Create scratch directory for temporary files
export SCRATCH="/home/v597/v597521/multileaspy"
mkdir -p $SCRATCH/$SLURM_JOB_ID

#Run job
srun --ntasks=$SLURM_NTASKS python /home/v597/v597521/multileaspy/
   multileaspy_par_mpi.py

#Remove files from scratch directory
rm -rf $SCRATCH/$SLURM_JOB_ID
```

Código 2: Código .sh para lanzar un trabajo en python en Magerit-3. Se solicitan un total de 40 tareas distribuidas en 1 único nodo. El tiempo máximo de ejecución del trabajo es de 50 horas.

Referencias

[1] Lisandro Dalcin et al. mpi4py: MPI for Python. mpi4py, 2023. Version 3.1.4.

[2] Hao Yu. Rmpi: Interface (Wrapper) to MPI (Message-Passing Interface). CRAN, 2023. R package version 0.7-2.